

Informe técnico practica intermedia

Juan Pablo Parra

Johan Peralta

Alessandro Soccol

Análisis y Diseño de Algoritmos, Universidad EAFIT

Ingeniería de Sistemas

Doc. Carlos Alberto Álvarez Henao

2026-2

Introducción

El propósito de este informe será explicar la practica intermedia del proyecto del curso de análisis y diseño de algoritmos, el mismo consistiendo en dos módulos, el primero basado en la prueba de todas las combinaciones posibles para encontrar el hash correspondiente a una contraseña y el segundo en enumerar cuales son las contraseñas posibles que cumplan una determinada política.

El siguiente documento estará estructurado entre las siguientes 12 secciones a través de las cuales diseccionaremos la construcción de ambas soluciones:

- Contextualización de los problemas presentados
- Fundamentación teórica en que se basa cada solución
- Modelamiento de las soluciones propuestas
- Diseño Algorítmico de dichas soluciones
- Seudocódigos de ambos programas
- Decisiones relevantes durante la implementación del código
- Análisis de la complejidad espacio - temporal de ambos módulos
- Resultados de la experimentación de ambos programas
- Casos de prueba para ambos códigos
- Análisis de los resultados obtenidos
- Comparación algorítmica entre ambas soluciones propuestas
- Conclusiones obtenidas a través de nuestros procedimientos

Este trabajo fue desarrollado en conjunto por Juan Pablo Parra, encargado del modulo de fuerza bruta, Johan Peralta, encargado del módulo de Backtracking y Alessandro Soccol, tester y encargado de la redacción del informe.

Contexto del problema (secc 4 y 5)

Modulo Fuerza Bruta: Para este problema nos tendríamos que situar en la piel de un atacante el cual, únicamente conociendo los hashes (SHA-256) de una contraseña, su longitud y el alfabeto de esta, tendría que hallar la correspondiente a base de probar todas y cada una de combinaciones hasta encontrar el objetivo, calculando sus hashes correspondientes y comparándolas con la base.

Modulo Backtracking: En este caso, tendríamos que situarnos como el desarrollador de políticas para contraseñas, el programa generaría una serie de contraseñas de forma incremental, validando en cada paso que la contraseña construida lleve a alguna solución posible y, en caso de que dejen de hacerlo, descartar las posibles contraseñas que se desarrollen a partir de ese punto.

Fundamentación teórica

Fuerza Bruta

- **Definición del paradigma:** Un algoritmo de fuerza bruta genera todos los posibles resultados para una solución, en nuestro caso, son todas las posibles contraseñas a partir entre un número mínimo y máximo de elementos y validadas al final contra un hash
- **Espacio de búsqueda:** Es la suma de todas las posibles combinaciones del alfabeto usado desde el mínimo de longitud hasta su máximo.
- **Enumeración exhaustiva:** Para que cada elemento se genere una sola vez, el código utiliza la función *incrementar_candidato* que se encarga de pasar al siguiente dígito posible para variar la contraseña.
- **Condiciones de aplicabilidad:** La longitud máxima de la contraseña debe ser mayor a 0 y a la mínima, el SHA-256 debe tener 64 caracteres hexadecimales, el alfabeto NO puede incluir dígitos repetidos y el espacio de búsqueda debe ser menor al máximo número permitido dentro de `uint64_t`
- **Complejidad temporal:** Para este caso la complejidad temporal corresponde a la sumatoria del alfabeto elevado por cada posible iteración desde el mínimo hasta el máximo, $O(|\Sigma|^n)$
- **Complejidad espacial:** La complejidad espacial por su lado es únicamente cada candidato probado a la vez, por lo que sería de orden $O(n)$
- **Ventajas:** Al ser un algoritmo de fuerza bruta, tenemos la total certeza de que el 100% de las ocasiones, si la petición es válida, encontrará la solución.
- **Limitaciones:** Debido a sus características, no guarda ninguna información previa que pueda ser útil para la construcción de una solución más eficiente, por lo que crece siempre de forma exponencial en base a la
- **Relaciones tamaño-coste:** El costo siempre crecerá exponencialmente en base a la longitud de la contraseña

Backtracking

- **Definición:** El código construye candidatos de forma incremental mediante *resolver_estado* y *explorar*, tras esto en cada paso, se dedica a verificar si la solución es factible o no, de no ser así, la función *es_factible* retorna falso y se impide que se siga construyendo dicha solución, implementando las máximas características del Backtracking que son la construcción incremental y la poda.
- **Relación con la búsqueda exhaustiva:** El código permite no ejecutar la poda si el último dígito que ingresas dentro del comando es 0, lo que hace que el programa recorra todos los nodos y posibles soluciones sin excepción, actuando del mismo modo que fuerza bruta.
- **Construcción incremental de soluciones:** Cada solución se va generando paso a paso y validando que se cumpla siempre las condiciones impuestas dentro de la política.

- **Representación del estado:** Los estados son cada candidato parcial que vayamos creando. Para nuestro caso, los estados se representan dentro de la tupla de *crear_clave*, en donde se van generando las contraseñas parciales.
- **Condiciones de factibilidad:** Se considerará no factible una solución que contenga 2 repetidos si la política indica que no se permiten, y si el numero de caracteres especificados faltantes son mayores a los restantes según lo especificado en *es_factible*.
- **Poda:** El código permite omitir la creación de soluciones no validas registrando dicho nodo como visitado, pero sin ejecutar el ciclo que permite la recursividad.
- **Árbol de búsqueda:**
- **Diferencia entre explorar todo el espacio y podar:** Mediante la poda, el algoritmo no tiene que generar contraseñas que desde un principio se sabe que no van a dirigirse hacia una solución valida, esto lo diferencia de explorar todo el espacio, donde incluso si supiéramos que una solución parcial y por ende todas las que se desprenden de ellas son invalidas, de igual forma tendríamos que generarlas antes de asegurarnos.
- **Complejidad temporal y espacial:** La complejidad temporal para el peor caso sigue siendo la misma que en fuerza bruta, es decir, tener que visitar todos los nodos posibles, por lo que es del orden $O(|\Sigma|^n)$, mientras que la espacial, aunque en teoría también se comparta con Fuerza bruta, es decir $O(n)$, en la practica esta siempre tendera a ser menor ya que gracias a la poda, se evitan generar contraseñas que se saben inviables con la política.
- **Mejor caso, caso promedio y peor caso:** El mejor caso posible para este tipo seria uno que, debido a las condiciones de su política, no tenga solución, el algoritmo de este modo podara todo el árbol de búsqueda antes de si quiera poder ejercer la recursividad y terminando instantáneamente. Sin embargo, para un caso promedio, no serán todas las ramas las que se poden, sino unas cuantas que dependerán de la especificidad de la política dada. Mientras que, en el peor de los casos, debido a una política muy vaga, el algoritmo tenga que recorrer todas y cada una de las posibles soluciones antes de poder encontrar una invalida.

Modelamiento

Fuerza Bruta

El espacio de búsqueda propuesto para la solución del módulo de fuerza bruta corresponde a la suma del alfabeto utilizado (A2: letras de la A a la Z y números del 0 al 9, aunque el mismo puede variar según lo que le indique el usuario, pudiendo ser más o menos símbolos), elevado desde la mínima longitud posible hasta la máxima longitud posible, a excepción que se sepa la longitud exacta en cuyo caso solo será el alfabeto usado elevado a la longitud de la contraseña. Esto ya que debe realizar cada posible contraseña para cada longitud posible.

Backtracking

Para Backtracking, en general, el espacio de búsqueda se calcula del mismo modo que en Fuerza Bruta, ya que lo único que cambia es la poda de soluciones inviables.

En este caso, para el módulo de Backtracking depende de la política que le defina el usuario durante su ejecución, si una de las variables mínimas es igual a 0, la misma no se contara para el alfabeto usado, pero en un caso ideal seria la suma del abecedario completo en ingles de 26 letras sumado tanto para minúsculas como mayúsculas, los números (del 0 al 9) y 5 caracteres especiales. Esto elevado por cada iteración desde 0 hasta la longitud fija de la contraseña, misma que se debe encontrar entre 1 y 10.

Diseño algorítmico

Fuerza bruta

Para este código es importante destacar el uso de la función *incrementar_candidato* como un contador que trabaja sobre la última solución generada, encargándose de este modo que no se repitan las contraseñas y garantizando la exploración completa del espacio de búsqueda. Además, se implementó una validación dentro de *calcular_espacio_busqueda* para prevenir el Overflow de `uint64_t` antes de que se empiece a correr para evitar fallas. Se opto además por implementar la librería PicoSHA2 para poder realizar las operaciones con código hash como las converticiones de las contraseñas generadas a código hash de este modo no se tendrían que implementar los hashes a mano.

En si el algoritmo para las pruebas de contraseña se basa en que, si el número mínimo y máximo de la longitud de la posible contraseña son distintos, entonces el código generara cada combinación del alfabeto otorgado para cada número entre la longitud mínima y máxima, guardando los resultados de cada ataque y el tiempo que se tardo en hallar la contraseña para ser usadas posteriormente en la gráfica de resultados.

Backtracking

Para Backtracking, cada contraseña se va a construir a partir de la política establecida desde el momento en que se ejecuta el comando y cada solución construida es evaluada en base a si sigue siendo factible o una solución valida (*es_factible* o *es_solucion*), lo que permite realizar la poda correspondiente en cuanto se sepa que la solución a la que se dirige no es válida. Dentro de *es_solucion*, además, se valida mediante la variable *faltantes* misma que se compara contra *restantes*, es decir, los caracteres que faltan por colocar, misma que hace que la cota sea mas barata de calcular en cada nodo.

Para desarrollar el Backtracking, además, se decidió imponer un rango de longitud de la contraseña de 1 hasta 10 debido a que el problema deja de ser computacionalmente tratable. Al momento de realizar la prueba con $n=4$ y asumiendo un alfabeto completo (de 67 caracteres en total) y omitiendo la poda, el programa tendría que

recorrer $\sum 67^4$, un total de 20,456,441 nodos y tardó unos 2,322,312 segundos en ejecutarse por completo, lo que en total nos da como resultado unos 8'808.653 nodos por segundo los que puede ejecutar una computadora promedio. Si tuviéramos que resolver una contraseña de $n=20$, por ejemplo, tendríamos que explorar aproximadamente 3.32×10^{36} nodos, y si esto lo dividimos entre la potencia de 8'808.653 nodos por segundo, nos estaríamos tardando aproximadamente 3.77×10^{29} segundos, o en términos de años 1.2×10^{22} años

```
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_p1 bt 4 3 2 2 1 0
metodo=bt_sin_poda
nodos_visitados=20456441
nodos_generados=20456440
soluciones_encontradas=0
tiempo_us=2322312
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol>
```

Seudocódigos

Ambos pseudocódigos se encuentran en la raíz base del repositorio como PSEUDOCODIGO_BT.md para fuerza bruta y PSEUDOCODIGO_BT.md para Backtracking.

Fuerza Bruta:

```
ALGORITMO FuerzaBrutaSHA256(H, A, m, n)
  VALIDAR que H sea un SHA-256 hexadecimal de 64 símbolos
  VALIDAR que A no este vacío y no tenga símbolos repetidos
  VALIDAR  $1 \leq m \leq n$ 

  intentos <- 0
  inicio <- RelojMonotono()

  PARA longitud <- m HASTA n HACER
    dígitos <- vector de longitud posiciones inicializadas en 0
    candidato <- cadena de longitud copias de A[0]

    REPETIR
      intentos <- intentos + 1
      SI SHA256(candidato) = H ENTONCES
        tiempo <- Microsegundos(RelojMonotono() - inicio)
        RETORNAR (VERDADERO, candidato, intentos, tiempo)
      FIN SI
    HASTA QUE IncrementarBaseA(dígitos, candidato, A) = FALSO
  FIN PARA

  tiempo <- Microsegundos(RelojMonotono() - inicio)
  RETORNAR (FALSO, cadena_vacia, intentos, tiempo)
FIN ALGORITMO
```

```
SUBALGORITMO IncrementarBaseA(dígitos, candidato, A)
  PARA posición <- longitud(dígitos) - 1 HASTA 0 PASO -1 HACER
    SI dígitos[posición] + 1 < longitud(A) ENTONCES
      dígitos[posición] <- dígitos[posición] + 1
      candidato[posición] <- A[dígitos[posición]]
    RETORNAR VERDADERO
  FIN SI

  dígitos[posición] <- 0
  candidato[posición] <- A[0]
  FIN PARA
  RETORNAR FALSO
FIN SUBALGORITMO
```

```
ALGORITMO AtaqueDiccionarioSHA256(H, ruta)
  VALIDAR que H sea un SHA-256 hexadecimal de 64 símbolos
  ABRIR archivo ubicado en ruta
  intentos <- 0
  inicio <- RelojMonotono()

  PARA CADA línea EN archivo, conservando su orden HACER
    candidato <- línea sin terminador CR/LF
    intentos <- intentos + 1
    SI SHA256(candidato) = H ENTONCES
      tiempo <- Microsegundos(RelojMonotono() - inicio)
      RETORNAR (VERDADERO, candidato, intentos, tiempo)
    FIN SI
  FIN PARA

  tiempo <- Microsegundos(RelojMonotono() - inicio)
  RETORNAR (FALSO, cadena_vacia, intentos, tiempo)
FIN ALGORITMO
```

Backtracking:

```

ALGORITMO BacktrackingConPoda(P, Σ)
  nodos_visitados <- 0
  soluciones <- 0
  actual <- cadena_vacia
  inicio <- RelojMonotono()

  PROCEDIMIENTO Resolver(actual)
    nodos_visitados <- nodos_visitados + 1

    // Evaluación de Factibilidad sobre el prefijo parcial
    SI NOT EsFactible(actual, P) ENTONCES
      RETORNAR // Poda efectiva de la rama
    FIN SI

    SI Longitud(actual) = P.n ENTONCES
      soluciones <- soluciones + 1
      RETORNAR
    FIN SI

    PARA CADA símbolo c EN Σ HACER
      actual.Agregar(c) // Aplicar selección
      Resolver(actual) // Llamada recursiva
      actual.RemoverUltimo() // Deshacer (Backtrack)
    FIN PARA
  FIN PROCEDIMIENTO

  Resolver(actual)
  tiempo <- Microsegundos(RelojMonotono() - inicio)
  RETORNAR (nodos_visitados, soluciones, tiempo)
FIN ALGORITMO

---

## Algoritmo 2: Función de Evaluación de Factibilidad

FUNCIÓN EsFactible(prefijo, P)
  k <- Longitud(prefijo)

  // Restricción local: No caracteres idénticos consecutivos
  SI k > 1 Y prefijo[k-1] = prefijo[k-2] ENTONCES
    RETORNAR FALSO
  FIN SI

  minus <- ContarMinúsculas(prefijo)
  mayus <- ContarMayúsculas(prefijo)
  dig <- ContarDigitos(prefijo)
  simb <- ContarSímbolos(prefijo)

  restantes <- P.n - k

  falta_minus <- Max(0, P.min_lower - minus)
  falta_mayus <- Max(0, P.min_upper - mayus)
  falta_dig <- Max(0, P.min_digit - dig)
  falta_simb <- Max(0, P.min_symbol - simb)

  total_faltantes <- falta_minus + falta_mayus + falta_dig + falta_simb

  // Poda por insuficiencia de casillas libres
  SI total_faltantes > restantes ENTONCES
    RETORNAR FALSO
  FIN SI

  RETORNAR VERDADERO
FIN FUNCIÓN

```

Implementación

Fuerza bruta

```

bool incrementar_candidato(std::vector<std::size_t>& digitos,
                          std::string& candidato,
                          const std::string& alfabeto) {
  for (std::size_t posicion = digitos.size(); posicion > 0; --posicion) {
    const std::size_t indice = posicion - 1;
    if (digitos[indice] + 1 < alfabeto.size()) {
      ++digitos[indice];
      candidato[indice] = alfabeto[digitos[indice]];
      return true;
    }

    digitos[indice] = 0;
    candidato[indice] = alfabeto.front();
  }
  return false;
}

std::uint64_t calcular_espacio_busqueda(const std::string& alfabeto,
                                        std::size_t longitud_minima,
                                        std::size_t longitud_maxima) {
  validar_alfabeto(alfabeto);
  validar_longitudes(longitud_minima, longitud_maxima);

  const auto base = static_cast<std::uint64_t>(alfabeto.size());
  const auto maximo = std::numeric_limits<std::uint64_t>::max();
  std::uint64_t potencia = 1;
  std::uint64_t total = 0;

  for (std::size_t longitud = 1; ++longitud) {
    if (potencia > maximo / base) {
      throw std::overflow_error("El espacio de búsqueda excede uint64_t.");
    }
    potencia *= base;
    total += potencia;
  }
  return total;
}

ComparacionAtaques comparar_ataques(const std::string& hash_objetivo,
                                     const std::string& alfabeto,
                                     std::size_t longitud_minima,
                                     std::size_t longitud_maxima,
                                     const std::string& ruta_diccionario) {
  ComparacionAtaques comparacion;
  comparacion.fuerza_bruta = ataque_fuerza_bruta(
    hash_objetivo, alfabeto, longitud_minima, longitud_maxima);
  comparacion.diccionario =
    ataque_diccionario(hash_objetivo, ruta_diccionario);
  return comparacion;
}

// namespace fb

```

Backtracking

```
private:
    const PoliticaConfig& politica;
    bool poda_activa;
    std::unordered_map<std::uint64_t, MetricasSubarbol> memoria;

    void validar_configuracion() const {
        if (politica.longitud == 0 || politica.longitud > 10) {
            throw std::invalid_argument("La longitud de BF debe estar entre 1 y 10.");
        }
        if (politica.min_minusculas < 0 || politica.min_mayusculas < 0 ||
            politica.min_digitos < 0 || politica.min_simbolos < 0) {
            throw std::invalid_argument("Los mínimos de la politica no pueden ser negativos.");
        }
    }
}
```

```
bool es_factible(const std::string& actual, int minusculas, int mayusculas,
                int digitos, int simbolos, bool tiene_repeticion) const {
    if (politica.prohibir_consecutivos_repetidos && tiene_repeticion) {
        return false;
    }

    const int restantes = static_cast<int>(politica.longitud - actual.size());
    const int faltantes =
        std::max(0, politica.min_minusculas - minusculas) +
        std::max(0, politica.min_mayusculas - mayusculas) +
        std::max(0, politica.min_digitos - digitos) +
        std::max(0, politica.min_simbolos - simbolos);
    return faltantes <= restantes;
}
```

Análisis de complejidad (secc 6)

Fuerza Bruta

- **Complejidad temporal:** El programa se ejecutará en bucle desde su longitud mínima (n_{min}) hasta la máxima (n_{max}) según se lo especifique el usuario, esto del modo $\sum_{n=n_{min}}^{n_{max}} n$. Debido a que fuerza bruta generará todas las posibles combinaciones del alfabeto en un espacio determinado y a que nuestro espacio corresponde a n , esto se representaría de la forma $\sum_{n=n_{min}}^{n_{max}} n * a^n$ donde a representa al alfabeto. Resultando de este modo la cantidad de contraseñas que se deberán de generar por cada longitud, puesto en términos de notación $O(n * m^n)$ y al ser una multiplicación, nos quedamos únicamente con el mayor factor que es $O(m^n)$. Sin embargo, para el caso particular del diccionario, debido a que no realiza ninguna función más allá de la comparación, entonces su complejidad espacial depende únicamente del orden $O(n)$.
- **Complejidad espacial:** El programa está ejecutando únicamente un candidato a la vez, sin realizar operaciones paralelas, lo procesa, verifica y lo valida o descarta, por este motivo, por iteración únicamente ocupara el equivalente al espacio de la solución construida, es decir, del orden $O(n)$. Para el caso particular del diccionario, ya se tiene una longitud fija del largo de las contraseñas, misma que no varía tras su ejecución, razón por la cual únicamente tiene una complejidad de $O(1)$.

Backtracking

- **Complejidad temporal:** Para este caso el alfabeto es una constante fija de 67 de longitud, aquí ya no podemos variar la longitud de la contraseña, así que se generaran de forma fija el alfabeto completo, un total de veces equivalente a la longitud n de la contraseña, dándonos como

resultado una complejidad temporal de $O(k^n)$, lo que implica que cada que se sube el número de n , el árbol de decisiones generara 67 nuevos nodos por cada nivel previo existente.

- **Complejidad espacial:** A pesar de que el árbol contenga k^n nodos en total, el algoritmo no lo retiene completo en memoria, sino que únicamente almacena la contraseña construida incrementalmente hasta el momento, razón por la cual su complejidad espacial corresponde directamente al largo de esta contraseña, es decir $O(n)$
-

Casos de prueba

Instancias modulo FB

- **Instancia de referencia:**

```
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_pl fb 8d51feb34e3e69f6fa6dffc577e2c68490cf9a7fcd835f9f6af1505b71d74773 abcdefghijklmnopqrstuvwxyz0123456789 5 5
metodo=fuerza_bruta
encontrada=si
contrasena=abc12
intentos=50249
tiempo_us=219315
```

- **Instancias por equipo/ Métricas a registrar por instancia:** Se encuentra dentro de results/tiempos_fb
- **Diccionario sintético:**

```
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_pl comparar 8d51feb34e3e69f6fa6dffc577e2c68490cf9a7fcd835f9f6af1505b71d74773 abcdefghijklmnopqrstuvwxyz0123456789 5 5 resources/diccionario.txt
metodo=fuerza_bruta
encontrada=si
contrasena=abc12
intentos=50249
tiempo_us=224282
---
metodo=diccionario
encontrada=no
contrasena=
intentos=500
tiempo_us=3997
```

Instancias del Módulo BT

- **Alfabeto:** Se uso un alfabeto de 67 caracteres en total en lugar de 69 evitando incluir la letra ñ, quedando del modo "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#\$%";
- **Parámetros de la política por equipo:**

```
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./verificar_semilla
VERIFICACION DE SEMILLA Y PARAMETROS BT
=====
Cadena concatenada: parraelmasriperaltabedoyasoccolmejia
Suma ASCII Total: 3817
SEMILLA OFICIAL: 3817
=====
PARAMETROS DE LA POLITICA BT (n = 8):
- minLower = 2 + (3817 % 2) = 3
- minUpper = 1 + (3817 % 2) = 2
- minDigit = 1 + (3817 % 3) = 2
- minSymbol = 1 (Fijo)
-----
Estado: VALIDO - Cumple minLower + minUpper + minDigit + minSymbol <= 8
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_pl bt 8 3 2 2 1 1
metodo=bt_con_poda
nodos_visitados=48580033119061
nodos_generados=48580033119060
soluciones_encontradas=9362522395200
tiempo_us=1
```

- **Variantes de dificultad:**

```
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_p1 bt 8 3 2 2 1 1
metodo:bt_con_poda
nodos_visitados=48580033119061
nodos_generados=48580033119060
soluciones_encontradas=9362522395200
tiempo_us=-1
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_p1 bt 6 3 2 2 1 1
metodo:bt_con_poda
nodos_visitados=1
nodos_generados=0
soluciones_encontradas=0
tiempo_us=0
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_p1 bt 10 3 2 2 1 1
metodo:bt_con_poda
nodos_visitados=680706374452025441
nodos_generados=680706374452025440
soluciones_encontradas=207096859600519200
tiempo_us=-1
PS C:\Users\alsoc\OneDrive - Universidad EAFIT\Semestre 5\Análisis y Diseño de Algoritmos\ADA_P1_Parra_Peralta_Soccol> ./ada_p1 bt 8 1 0 0 1 1
metodo:bt_con_poda
nodos_visitados=376741790569851
nodos_generados=376741790569850
soluciones_encontradas=358778326970752
tiempo_us=-1
```

- **Instancia de referencia común:**
- **Métricas para registrar por instancia:** Se encuentran dentro de results/tiempo_bt

Experimentación

Toda la información correspondiente a este modulo se encuentra dentro de la carpeta results del repositorio

Resultados

Fuerza Bruta

instancia	tipo	contraseña_real	hash_sha256	alfabeto_tam	longitud	intentos_fb	tiempo_fb_us	tiempo_fb_ms	encontrada_dict	intentos_dict	tiempo_dict_us	tiempo_dict_ms
equipo_1	equipo	rmmb	64b2c55404531af2a0256884c424292f5c7d973f96ce59bc1440d4aa866d7fe	26	4	238188	228379	228.379	no	500	6591	6.591
equipo_2	equipo	4tmm	850d496e723bc12906c54830b6c2b36ef74c44139970bxc0bb7e367aebba	36	4	1472742	1579201	1579.201	no	500	569	0.569
equipo_3	equipo	rffgds	0ca73cdeb6f6abc936f3a5aa7d08563cc4231072ea03189a286b393828147e	26	5	6050999	10001988	10001.988	no	500	1070	1.070
equipo_4	equipo	1qbgth	f7593daa77fa5f53a8e2417a665635cd8024e3c5f9f6ae10f96dcd305a90	36	5	47825252	79234382	79234.382	no	500	2012	2.012
equipo_5	equipo	mpkffex	18ed8547e3382a2be7015025e2d5658ad8afaa3d07a3521d297e99cd127bc776	26	6	161967518	224351529	224351.529	no	500	690	0.690
calibracion_a1_n3	calibracion	abc	ba7816bf0f1cfea4141403e5dae2223600361a396177ab6410ff61f20015ad	26	3	731	746	0.746	no	500	616	0.616
calibracion_a2_n3	calibracion	abc	ba7816bf0f1cfea4141403e5dae2223600361a396177ab6410ff61f20015ad	36	3	1371	1649	1.649	no	500	613	0.613

Backtracking

	variante	longitud	k	poda_activa	nodos_visitados	nodos_generados	soluciones	medicion	tiempo_us	tiempo_ms	porcentaje_reduccion
1	equipo_n8	8	8	True	48580033119061	48580033119060	9362522395200	cota_teorica_timeout	-1	-1.0	88.215
2	equipo_n8	8	8	False	412220218125681	412220218125680	9362522395200	cota_teorica_timeout	-1	-1.0	0.0
3	equipo_n6	6	8	True	1	0	0	empirica_enumerativa	0	0.0	0.0
4	equipo_n6	6	8	False	91828963717	91828963716	0	empirica_enumerativa	1241421280	1241421.28	0.0
5	equipo_n10	10	8	True	680706374452025441	680706374452025440	207096859600519200	cota_teorica_timeout	-1	-1.0	63.214
6	equipo_n10	10	8	False	1850456559166182077	1850456559166182076	207096859600519200	cota_teorica_timeout	-1	-1.0	0.0
7	relajada_n8	8	1	True	376741790569851	376741790569850	358778326970752	cota_teorica_timeout	-1	-1.0	8.607
8	relajada_n8	8	1	False	412220218125681	412220218125680	358778326970752	cota_teorica_timeout	-1	-1.0	0.0
9	sin_restricciones_n6	6	0	True	91828963717	91828963716	90458382169	empirica_enumerativa	760922883	760922.883	0.0
10	sin_restricciones_n6	6	0	False	91828963717	91828963716	90458382169	empirica_enumerativa	679341425	679341.425	0.0
11	calibracion_n4_1	4	4	True	5327841	5327840	811200	empirica_enumerativa	39398	39.398	0.0
12	calibracion_n4_1	4	4	False	20456441	20456440	811200	empirica_enumerativa	145199	145.199	0.0
13	calibracion_n4_2	4	4	True	7691668	7691667	2068560	empirica_enumerativa	62259	62.259	0.0
14	calibracion_n4_2	4	4	False	20456441	20456440	2068560	empirica_enumerativa	146795	146.795	0.0
15	calibracion_n4_3	4	3	True	8102981	8102980	1427400	empirica_enumerativa	62312	62.312	0.0
16	calibracion_n4_3	4	3	False	20456441	20456440	1427400	empirica_enumerativa	142231	142.231	0.0
17	calibracion_n4_4	4	3	True	13851045	13851044	3062280	empirica_enumerativa	108833	108.833	0.0
18	calibracion_n4_4	4	3	False	20456441	20456440	3062280	empirica_enumerativa	146171	146.171	0.0
19	calibracion_n4_5	4	0	True	19854915	19854914	19262232	empirica_enumerativa	160571	160.571	0.0
20	calibracion_n4_5	4	0	False	20456441	20456440	19262232	empirica_enumerativa	150968	150.968	0.0

Análisis de resultados

Fuerza Bruta

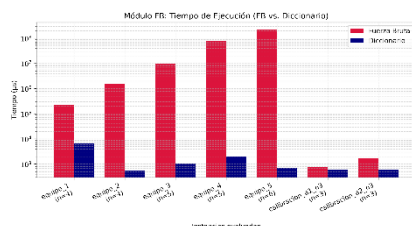
Según los resultados de la sección anterior podemos confirmar un crecimiento exponencial para cada aumento de la longitud de la contraseña, como se puede evidenciar en las pruebas donde, mientras para $n=3$ la respuesta tarda 746 microsegundos, para cuando la incrementamos a $n=6$ tardara 224351529 microsegundos. Por otro lado, según la grafica de crecimiento exponencial muestra una tendencia irónicamente logarítmica, pero esto bien se puede deber al reducido tamaño de la muestra, debido a que el resto de los datos así lo indica. Así mismo vale la pena rescatar que, para la comparativa entre FB vs diccionario, este ultimo tiende a tener valores mucho mas regulares y reducidos en comparación con fuerza bruta, que crece de forma exponencial, sin embargo, según lo propio que vimos en las pruebas, diccionario tiende a no encontrar la respuesta correcta debido a que depende de que el propio hash traducido se encuentre allí, mientras que fuerza bruta, si se trata de una solución posible, siempre lograra encontrarla

Backtracking

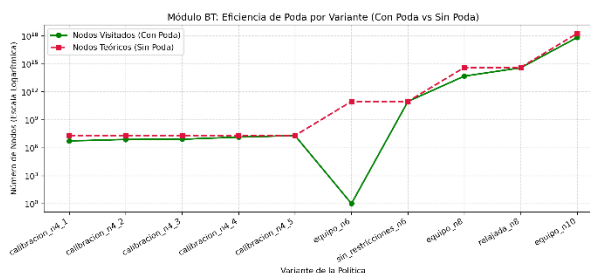
De lo visto dentro de los resultados se evidencia la importancia de la poda para este tipo de algoritmos, logrando hacer una comparación empírica entre el total de nodos sin poda y los nodos con poda incluida, así mismo, dadas las características del ejercicio, se muestra como algunos resultados tienen tiempo de -1, esto debido, a que se usa `resolver()` estima cuantos nodos tendrá que visitar y si el mismo supera al `LIMITE_INTERACTIVO_GLOBAL` definido en main, entonces el programa no resuelve la `resolver_enumerativo()` y retorna por defecto -1.

Comparación algorítmica

FB vs diccionario



BT con poda vs sin poda



Conclusiones

Tras la realización de ambos códigos y con los resultados de las pruebas correspondientes podemos concluir que:

- Ambos programas, aunque efectivos en encontrar su objetivo, son intratables para casos generales debido a los tiempos de espera tan elevados.
- La efectividad de la poda en Backtracking siempre va a depender de la especificidad de las restricciones que se le impongan
- Para la ejecución de este tipo de códigos se requieren computadoras de muy alta potencia si se desean obtener tiempos razonables, por ejemplo, mientras que la computadora usada para los primeros análisis del programa podía visitar de 8.8 nodos por segundo, algunas de los que se usaron para el desarrollo del programa fueron incluso más rápidas y eficientes a la hora de obtener resultados.
- Es importante realizar validaciones tempranas dentro del código para evitar tantos errores costos en instancias futuras de la ejecución.

Referencias

- Okada S, Klitzke E, Cavalier F, Sigh (2025) PicoSHA2 [<https://github.com/okdshin/PicoSHA2>]
- CSV viewer & editor online. <https://csv-viewer-online.github.io/>

Uso de herramientas de IA

- <https://claude.ai/share/34cae796-e91c-4557-b87b-958f65c48eb1>
- Archivo adjunto dentro de report/PROMTS.docx

Contribución individual de los integrantes

- Juan Pablo Parra se encargó del desarrollo completo del modulo de Fuerza bruta, así como del pseudocódigo correspondiente al mismo y el archivo README indicando las instrucciones base para la implementación de su parte del código.
- Johan Peralta se encargó del modulo de Backtracking completo, incluyendo su pseudocódigo y las debidas actualizaciones del README, así como de la creación del propio repositorio y su estructura base.
- Alessandro Soccol se encargó de la redacción del informe, así como de la realización individual de pruebas de los códigos presentados para verificar su correcto funcionamiento.